

SOAL	JAWABAN
Return first_name, last_name, and salary for employees in IT (department 60) or Sales (department 80) earning more than 6000.	select first_name, last_name, salary from employees where (department_id = 60 or department_id = 80) and salary > 6000;
Return first_name, last_name, department_id, and salary for employees who are either in department 90 or earn more than 12000.	select first_name, last_name, department_id, salary from employees where department_id = 90 or salary > 12000;
From the store sandbox, return first_name, last_name, and country for customers whose country is not 'USA', ordered by country, then last_name.	select first_name, last_name, country from customers where country <> 'USA' order by country, last_name;
From the products table (store sandbox), return name and price for products priced between 30 and 80 inclusive.	select name, price from products where price between 30 and 80;
Return first_name and last_name of employees whose first name starts with the letter A.	select e.first_name, e.last_name from employees e where e.first_name like 'A%';
From the hospital sandbox, return first_name and last_name of patients whose first_name starts with 'C' or 'D', ordered by first_name.	select first_name, last_name from patients where first_name like 'C%' or first_name like 'D%' order by first_name;
From the store sandbox, return name, category_id, and price for products where category_id is 1, 2, or 3. Use IN. Order by category_id, then name.	select name, category_id, price from products where category_id in (1, 2, 3) order by category_id, name;
Return first_name and last_name of HR employees who have NO manager (manager_id is NULL).	select first_name, last_name from employees where manager_id is null;
From the hospital sandbox, return first_name and last_name of patients whose gender is recorded (not NULL).	select first_name, last_name from patients where gender is not null;
Return first_name and last_name of employees who have both a manager_id and a department_id (neither is NULL). Order by last_name.	select first_name, last_name from employees where manager_id is not null and department_id is not null order by last_name;
From the store sandbox, return first_name and last_name of customers who have placed at least one order. Use EXISTS. Order by last_name.	select first_name, last_name from customers c where exists (select 1 from orders o where o.customer_id = c.id) order by last_name;
From the store sandbox, return first_name and last_name of customers who have never placed an order. Use NOT EXISTS. Order by last_name.	select first_name, last_name from customers c where not exists (select 1 from orders o where o.customer_id = c.id) order by last_name;
From the hospital sandbox, return first_name and last_name of patients who appear in the appointments table at least once. Use EXISTS. Order by last_name.	select first_name, last_name from patients p where exists (select 1 from appointments a where a.patient_id = p.id) order by last_name;
Return first_name and last_name of HR employees	select first_name, last_name from employees e1

who are the manager_id for at least one other employee. Use EXISTS. Order by last_name.	where exists (select 1 from employees e2 where e1.id = e2.manager_id) order by last_name;
Return a single column named total with the total number of employees.	select count(*) as total from employees;
From the store sandbox, return the average product price as a column called avg_price.	select avg(price) as avg_price from products;
Return max_salary and min_salary for all employees (using MAX and MIN).	select max(salary) as max_salary, min(salary) as min_salary from employees;
From the store sandbox, return the total revenue as total_revenue — the sum of quantity * unit_price across all order items.	select sum(quantity*unit_price) as total_revenue from order_items;
Return department_id and employee_count for every department in the HR sandbox.	select department_id, count(*) as employee_count from employees group by department_id;
From the store sandbox, return status and the number of orders (order_count) in each status.	select status, count(*) as order_count from orders group by status;
From the store sandbox, return category_id and product_count (the number of products in each category), ordered by category_id.	select category_id, count(*) as product_count from products group by category_id order by category_id;
From the hospital sandbox, return each appointment status and the number of appointments in that status (appt_count), ordered by status.	select status, count(*) as appt_count from appointments group by status order by status;
Return department_id and employee_count for departments with 5 or more employees.	select department_id, count(*) as employee_count from employees group by department_id having count(*) >= 5;
From the store sandbox, return category_id and avg_price (rounded to 2 decimal places) for product categories whose average price is above 50.	select category_id, round(avg(price), 2) as avg_price from products group by category_id having avg(price) > 50;
From the hospital sandbox, return each specialty and doctor_count for specialties that have more than 2 doctors, ordered by specialty.	select specialty, count(*) as doctor_count from doctors group by specialty having count(*) > 2 order by specialty;
Return a single value unique_departments — the number of distinct department_id values across all employees.	select count(distinct department_id) as unique_departments from employees;
From the store sandbox, return two columns (isinya total order masing2 status) in a single query: shipped (orders with status 'shipped') and cancelled (status 'cancelled'). Use FILTER.	select count(*) filter (where status = 'shipped') as shipped, count(*) filter (where status = 'cancelled') as cancelled from orders;
Return department_id, total employees, and high_earners (those with salary > 8000) per department. Use FILTER. Order by department_id.	select department_id, count(*) as total, count(*) filter (where salary > 8000) as high_earners from employees group by department_id order by department_id;

Return first_name, last_name, and the department name (alias department) for every employee that has a department.	select e.first_name, e.last_name, d.name as department from employees e join departments d on d.id = e.department_id;
Return order_id, product name (alias product), quantity, and unit_price for every order item.	select o.order_id, p.name as product, o.quantity, o.unit_price from order_items o join products p on o.product_id = p.id;
From the hospital sandbox, return patient first_name (alias patient_first), patient last_name (alias patient_last), doctor first_name (alias doctor_first), and doctor last_name (alias doctor_last) for every appointment.	select p.first_name as patient_first, p.last_name as patient_last, d.first_name as doctor_first, d.last_name as doctor_last from appointments a join doctors d on a.doctor_id = d.id join patients p on a.patient_id = p.id;
From the store sandbox, return product name (alias product), category name (alias category), and price, ordered by category, then product.	select p.name as product, c.name as category, p.price from products p join categories c on c.id = p.category_id order by category, product;
Return first_name and last_name of customers who have NO orders.	select c.first_name, c.last_name from customers c left join orders o on o.customer_id = c.id where o.id is null;
Return doctor first_name, last_name, and the number of appointments they have (alias appointments). Include doctors with zero appointments.	select d.first_name, d.last_name, coalesce(count(a.id), 0) as appointments from doctors d left join appointments a on a.doctor_id = d.id group by d.first_name, d.last_name;
Return every department name and the number of employees in it (employee_count), including departments with zero employees. Order by name.	select d.name, coalesce(count(e.id),0) as employee_count from departments d left join employees e on e.department_id = d.id group by d.name order by d.name;
From the store sandbox, return each product name and total_sold (sum of quantity from order_items). Include products that have never been sold (show 0). Order by total_sold descending.	select p.name, coalesce(sum(o.quantity),0) as total_sold from products p left join order_items o on o.product_id = p.id group by p.id, p.name order by sum(o.quantity) desc; // group by HARUS mengandung primary key
From the store sandbox, return customer id, first_name, last_name, and order_count (including 0 for customers with no orders). Use a RIGHT JOIN from orders to customers. Order by id.	select c.id as customer_id, c.first_name, c.last_name, coalesce(count(o.id), 0) as order_count from orders o right join customers c on o.customer_id = c.id group by c.id, c.first_name, c.last_name order by c.id;
From the HR sandbox, return job title and employee first_name + last_name using a FULL OUTER JOIN between jobs and employees on job_id. Show NULL where no match exists. Order by title NULLS LAST, then last_name NULLS LAST.	select j.title as job_title, e.first_name, e.last_name from jobs j full outer join employees e on e.job_id = j.id order by j.title nulls last, e.last_name nulls last;
From the hospital sandbox, FULL OUTER JOIN departments to doctors on department_id. Return department name and doctor first_name, last_name. Order by name NULLS LAST, last_name NULLS LAST.	select e.name as department, o.first_name, o.last_name from departments e full outer join doctors o on o.department_id = e.id order by e.name nulls last, o.last_name nulls last;

From the HR sandbox, return first_name, last_name, department name (alias department), and job title (alias job_title) for every employee. Order by last_name.	select e.first_name, e.last_name, d.name as department, j.title as job_title from employees e join departments d on e.department_id = d.id join jobs j on j.id = e.job_id order by e.last_name;
From the store sandbox, join orders, customers, order_items, and products. Return customer first_name, product name, quantity, and unit_price. Order by orders.id, then products.name.	select c.first_name, p.name as product, oi.quantity, oi.unit_price from orders o join order_items oi on oi.order_id = o.id join customers c on c.id = o.customer_id join products p on oi.product_id = p.id order by o.id, p.name;
From the hospital sandbox, join appointments, patients, doctors, and departments. Return patient first_name, doctor first_name, department name (alias department), and appointment_date. Order by appointment_date, then patients.last_name.	select p.first_name as patient, doc.first_name as doctor, de.name as department, a.appointment_date from appointments a join patients p on p.id = a.patient_id join doctors doc on doc.id = a.doctor_id join departments de on de.id = doc.department_id order by a.appointment_date, p.last_name; //hati-hati kasih alias, jangan pake do
Return first_name, last_name, and salary of employees earning strictly more than the company average salary.	select first_name, last_name, salary from employees where salary > (select avg(salary) from employees);
Return the name of every product that does NOT appear in any order_items row.	select name from products where id not in (select product_id from order_items);
Return first_name and last_name of customers who have placed at least one order. Use EXISTS. Order by last_name.	select first_name, last_name from customers c where exists (select 1 from orders o where o.customer_id = c.id) order by last_name;
From the store sandbox, return name, price, and category_id for products whose price is strictly above the average price in their own category. Order by category_id, then price descending.	select name, price, category_id from products p1 where price > (select avg(p2.price) from products p2 where p1.category_id = p2.category_id) order by category_id, price desc;
Using a CTE, return the first_name, last_name, and total_spent of the top 5 customers by total order-item value (quantity * unit_price). Order by total_spent descending.	with cte as (select o.customer_id, sum(oi.quantity * oi.unit_price) as total_spent from order_items oi join orders o on o.id = oi.order_id group by o.customer_id) select c.first_name, c.last_name, cte.total_spent from customers c join cte on cte.customer_id = c.id order by cte.total_spent desc limit 5;
Use a CTE named dept_stats to compute department_id , avg_salary (round(, 2)) and emp_count per department_id. Then return all columns from dept_stats ordered by avg_salary descending.	with dept_stats as (select department_id, round(avg(salary), 2) as avg_salary, count(id) as emp_count from employees group by department_id) select * from dept_stats order by avg_salary desc;
Use a CTE to count appointments per doctor, then join to departments. Return the top 3 department name values and their total appt_count, ordered by appt_count descending. //SOAL GAJELAS!!	WITH doc_appts AS (SELECT doc.department_id, COUNT(a.id) AS appt_count FROM doctors doc LEFT JOIN appointments a ON a.doctor_id = doc.id GROUP BY doc.department_id) SELECT d.name AS department, da.appt_count FROM doc_appts da JOIN departments d ON d.id = da.department_id

	ORDER BY da.appt_count DESC LIMIT 3;
Return first_name, last_name, salary, and department_id for employees whose salary exceeds the average salary in their own department. Use a correlated subquery. Order by department_id, then salary descending.	select first_name, last_name, salary, department_id from employees e1 where salary > (select avg(salary) from employees e2 where e1.department_id = e2.department_id) order by department_id, salary desc;
From the store sandbox, return name, price, and category_id for products whose price exceeds the average price in their own category. Use a correlated subquery. Order by category_id, price descending.	select name, price, category_id from products p where price > (select avg(price) from products p2 where p.category_id = p2.category_id) order by category_id, price desc;
From the hospital sandbox, return first_name and last_name of patients whose appointment count exceeds the average number of appointments per patient. Use a correlated subquery. Order by last_name.	select first_name, last_name from patients p1 where (select count(a1.id) from appointments a1 where a1.patient_id = p1.id) > (select avg(total) from (select a2.patient_id, count(a2.id) as total from appointments a2 group by a2.patient_id)) order by last_name;
Write two CTEs: dept_stats (avg salary and headcount per department) and ranked_depts (rank departments by avg salary descending). Return department name, avg_salary, and emp_count for all departments, ordered by rank.	with dept_stats as (select department_id, round(avg(salary), 2) as avg_salary, count(*) as emp_count from employees group by department_id), ranked_depts as (select *, rank() over (order by avg_salary desc) as rn from dept_stats) select d.name as department, rd.avg_salary, rd.emp_count from ranked_depts rd join departments d on d.id = rd.department_id order by rd.rn;
Use two CTEs: category_revenue (total revenue per category from order_items + products) and ranked_cats (rank by revenue descending). Return the top 3 category name and revenue, ordered by rank.	with category_revenue as (select p.category_id, sum(oi.quantity * oi.unit_price) as revenue from order_items oi join products p on p.id = oi.product_id group by p.category_id), ranked_cats as (select *, rank() over (order by revenue desc) as rn from category_revenue) select c.name as category, rc.revenue from categories c join ranked_cats rc on rc.category_id = c.id where rc.rn <= 3 order by rc.rn;
Use two CTEs: doc_stats (appointment count per doctor) and ranked_docs (rank doctors by appointment count descending). Return the top 5 doctor first_name, last_name, and total_appointments, ordered by rank.	with doc_stats as (select a.doctor_id, count(a.id) as total_appointments from appointments a group by a.doctor_id), ranked_docs as (select *, rank() over (order by total_appointments desc) as rn from doc_stats) select d.first_name, d.last_name, rd.total_appointments from doctors d join ranked_docs rd on rd.doctor_id = d.id where rd.rn <= 5 order by rd.rn;

Return first_name, last_name, department_id, salary, and a column dept_rank that ranks employees within each department by salary descending (use ROW_NUMBER).	select first_name, last_name, department_id, salary, row_number() over(partition by department_id order by salary desc) as dept_rank from employees;
Return first_name, last_name, department_id, salary, and salary_rank using RANK() (not ROW_NUMBER) within each department ordered by salary descending.	select first_name, last_name, department_id, salary, rank() over(partition by department_id order by salary desc) as salary_rank from employees;
Using a CTE with ROW_NUMBER(), return only the highest-paid employee per department. Show first_name, last_name, department_id, and salary, ordered by department_id.	with cte as (select *, row_number() over(partition by department_id order by salary desc) as rn from employees) select first_name, last_name, department_id, salary from cte where rn = 1 order by department_id;
From the store sandbox, return one row per order with order_date, id (order_id), and a running_orders column showing the cumulative count of orders up to and including that date, ordered by order_date then id.	select order_date, id, count(*) over(order by order_date, id) as running_orders from orders order by order_date, id;
Return first_name, last_name, hire_date, salary, and salary_diff — the difference between this employee's salary and the previous hire's salary (ordered by hire_date, then id). Use LAG.	select first_name, last_name, hire_date, salary, salary - lag(salary) over(order by hire_date, id) as salary_diff from employees order by hire_date, id;
From the store sandbox, return id (alias order_id), order_date, and next_order_date (the date of the following order when sorted by order_date, id). Use LEAD.	select id as order_id, order_date, lead(order_date) over(order by order_date, id) as next_order_date from orders;
Return first_name, last_name, salary, and quartile (1–4) using NTILE(4) ordered by salary ascending. Quartile 1 = lowest earners. Order by quartile, then salary.	select first_name, last_name, salary, ntile(4) over (order by salary asc) as quartile from employees order by quartile, salary;
Return first_name, last_name, salary, and pct_rank — the PERCENT_RANK() (multiplied by 100 and rounded to 2 decimal places) of each employee ordered by salary. Order by salary.	select first_name, last_name, salary, round((percent_rank() over (order by salary) * 100)::numeric, 2) as pct_rank from employees order by salary; //round cuma terima (numeric, integer) jadi typenya perlu disesuaikan dengan taro (..)::numeric //percent_rank() hasilnya nilai double precision
Using a CTE with NTILE(4), return only the employees in the top salary quartile (quartile = 4). Show first_name, last_name, and salary, ordered by salary descending.	with cte as (select *, ntile(4) over(order by salary) as quartile from employees) select first_name, last_name, salary from cte where quartile = 4 order by salary desc;
Return first_name, last_name, hire_date, salary, and running_total — a cumulative sum of salary ordered by hire_date, then id. Use ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW.	select first_name, last_name, hire_date, salary, sum(salary) over(order by hire_date, id rows between unbounded preceding and current row) as running_total from employees;

Return first_name, last_name, hire_date, salary, and moving_avg — the average salary across the current employee and 1 row before and 1 row after (ordered by hire_date, id). Round to 2 decimal places.	select first_name, last_name, hire_date, salary, round(avg(salary) over(order by hire_date, id rows between 1 preceding and 1 following), 2) as moving_avg from employees;
Return first_name, last_name, department_id, salary, and pct_of_dept — each employee's salary as a percentage of their department's total salary. Use ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING. Round to 2 decimal places. Order by department_id, salary descending.	select first_name, last_name, department_id, salary, round(salary/sum(salary) over(partition by department_id rows between unbounded preceding and unbounded following) * 100, 2) as pct_of_dept from employees order by department_id, salary desc;